Visual Question Answering From Charts

Nagesh Kunar*, Adarsh Ramesh Thimmapurmath†
*†School Of Computing, Dublin City University
Dublin, Ireland
Email: {nagesh.kumar2, adarshramesh.thimmapurmath2}@mail.dcu.ie

***Abstract*—Visual Question Answering (VQA) over charts requires understanding both visual and textual elements within data visualizations, which traditional VQA systems struggle with. Existing approaches like DVQA [3], FigureQA [4], and PlotQA [1] use synthetic charts with limited layout variability, while ChartQA [2] introduces real-world charts but often relies on OCR or multi-modal models with limited structural understanding. These methods struggle to accurately capture chart semantics like axis labels, titles, and legends. In contrast, we use YOLO to explicitly detect chart components, extract them as structured text, and combine them with the question and flattened table data. A T5 model then performs question answering in a pure text-to-text setting, enabling interpretable and structure-aware reasoning.**

We implemented and tested our system on Google Colab Pro with GPU acceleration using the ChartQA [2] training dataset, which includes four chart types: pie charts, line charts, vertical bar charts, and horizontal bar charts. The dataset supports a variety of question types, including numeric, alphabetic, and binary (yes/no) questions.

***Index Terms*—Chart VQA, Faster R-CNN, OCR, COCO Dataset, Visual Reasoning, YOLO, Text To Text Transfer Transformer (T5) .**

I. Introduction

We are in a data-driven generation wherein charts or plots act as primary tools to convey the complex datasets efficiently. Visualizations such as bar charts, line graphs, and pie charts help the scientist, financial dashboards, or journalists either to make data understandable. However, what might seem easy to grasp for a human reader may be a very complex task for machines to do meaning extraction from such visualizations. Chart Image VQA is an advancing area that combines computer vision, natural language understanding, and reasoning to answer natural language questions posed on data visualizations.

Natural images, where the content is either visual or semantic but not both, have been the primary focus of traditional VQA systems. Charts, on the other hand, are hybrid artifacts that include structured numerical and categorical data that has been visually encoded using textual labels, lines, bars, legends, and colors. As a result, a VQA system that targets charts needs to integrate features from tabular reasoning and image understanding.

To address this intersection, a number of benchmark datasets and models have been developed. While FigureQA focused on figure reasoning but used little real-world semantics, DVQA [3] introduced synthetic charts with programmatically generated questions. The field was advanced by PlotQA [1], which offered a variety of question types and large-scale real-world charts. Though it still relied on Vision-TAPAS for semantic understanding, which had trouble with highly visual tasks, ChartQA [2] further integrated visual and logical reasoning. More recent models, like DePlot [6] and MATCHA [7], used multimodal pretraining and one-shot plot-to-table translation, but they were sensitive to variations in chart layout and demanded a lot of processing power.

In order to overcome these constraints, we suggest a hybrid strategy that combines a language model (T5 [21]) to comprehend and reason over the extracted chart data with a deep learning-based object detection model (YOLO) to identify chart elements (axes, bars, legends, titles, etc.). Our approach first segments and extracts chart entities using YOLO's bounding box annotations, which are subsequently flattened into a structured table format. A T5-based model that has been adjusted for question answering is then fed this derived table.

At first, we looked into a CNN-based pipeline that was comparable to early vision architectures like AlexNet and VGG, but their slower inference and training overhead made them less appropriate for our real-time objectives. For effective component localization—which is essential for real-world deployment—YOLO offered quicker and more reliable detection.

We used datasets from HuggingFace's ChartQA and DVQA versions to develop and test our system on Google Colab Pro with GPU support. Multiple chart types and question formats, such as arithmetic, color-based, logical, and visual reasoning,

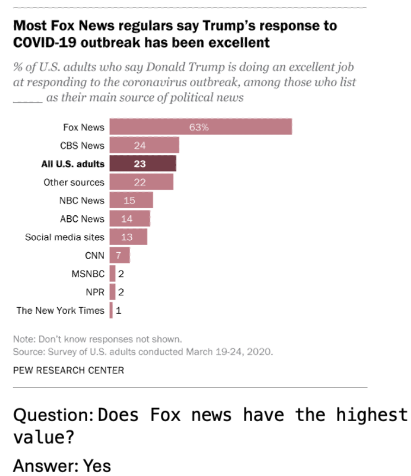

Fig. 1. Sample chart and QA pair in our dataset

are supported by the system. The findings show that our pipeline outperforms previous efforts in terms of speed and flexibility.

Our goal is to close the gap between structured reasoning and chart visual comprehension. This work will have implications for data analysis, automated report generation, and multimodal search interfaces in the future.

II. Literature Review

A. Dataset

Previous studies have emphasized advancing chart comprehension through datasets (shown in Table I) and models that address real-world complexity, such as handling out-of-vocabulary (OOV) answers, numerical reasoning, and multimodal integration. [1], [2], and [3] prioritize large-scale, diverse question-answer pairs to challenge models with tasks like arithmetic operations and chart-specific terminology. These datasets share a focus on bridging synthetic and real-world scenarios, though their implementations differ: PlotQA [1] leverages real-world data sources (e.g., World Bank) for numerical OOV challenges, while ChartQA [2] balances human-written analytical questions with machine-generated templated queries to ensure breadth and depth. [1] presents an extensive collection of 28 million question–answer pairs extracted from 224K scientific plots. This dataset is designed to challenge models with numerical reasoning and out-of-vocabulary answers. In a similar way, [2] compiles 20,882 charts accompanied by over 32,000 questions—crafted both by humans and generated automatically—to support tasks requiring arithmetic and logical reasoning. Focusing on bar charts, [3] assembles more than 3 million image–question pairs, which emphasize the need to process chart-specific text. Adding further diversity, [3] utilizes data from 52 real-world charts along with hundreds of human-generated questions and visual explanations, thereby enriching the dataset with interpretability aspects. Meanwhile, they leverages the ChartSFT dataset to cover multiple chart types, supporting tasks such as summarization and numerical reasoning. Complementary to these, Deplot and matcha build upon datasets like PlotQA and ChartQA to convert visual plots into structured tables and incorporate mathematical reasoning into the learning process. For retrieval and summarization tasks, [15] adapts existing chart datasets, and [9] offers a benchmark consisting of 44,096 charts for generating natural language summaries. Finally, although primarily text-based, [10] has significantly influenced multimodal research by providing over 100,000 question–answer pairs that set a high standard for precise span extraction.

TABLE I
COMPARISON BETWEEN EXISTING DATASETS

Dataset	Size	Types
PlotQA	224,377 plots	Line, bar, scatter plots
ChartQA	20,882 charts	Bar, line, pie
DVQA	3M image–question pairs	Bar charts only
Chart-to-Text	44,096 charts	Bar, line, pie, radar, box plots

B. Data Extraction

Previous studies by PlotQA [1] cannot be considered as conclusive because it leverages a TableQA framework to tackle open-vocabulary questions, yet it neglects the integration of visual chart elements—such as color and layout which are important for visual reasoning. As the authors note earlier, more work is necessary to unify data tables and visual attributes for tasks like arithmetic operations. This gap is addressed by [2] which explicitly incorporates visual references into benchmark, enabling models to reason over both data tables and visual attributes for tasks like arithmetic operations. Many models rely on structured data extraction (e.g., OCR, table parsing) to translate visual charts into machine-readable formats. Hybrid models [1] and [6] exemplify this approach, combining OCR with table extraction for QA(Question and Answer) tasks. However, innovations diverge in their emphasis: [7] integrates math reasoning pretraining to enhance numerical accuracy, whereas chart paper adopts a multitask framework for zero-shot generalization across chart types. Transformer-based architectures like T5 and VisionTaPas [2] unify visual and textual features, while [3] DVQA employs dynamic encoding to handle chart-specific vocabularies.

A recurring challenge across studies is the loss of visual context (e.g., color, spatial relationships) during data extraction, which impacts performance on tasks requiring color-based reasoning or positional analysis. While [6] DePLOT achieves high accuracy in table reconstruction (97.1% RNSS), it struggles with color-dependent queries, a limitation also noted in MATCHA [7]. Similarly, Chart-to-Text [16] and Text-to-Text [21] models generate fluent summaries but face hallucinations and factual inaccuracies, reflecting broader issues in balancing fluency and precision.

In contrast, SQuAD [10], though text-focused, underscores the universal gap between human and machine comprehension, with machines achieving 51% F1 versus humans at 86.8%. This mirrors challenges in chart VQA [3], where even state-of-the-art models like DePLOT + Codex (67.6% accuracy on ChartQA) fall short of human-like robustness.

The papers collectively advocate for multimodal fusion and specialized pretraining as pathways forward. Yet, their strategies vary: some prioritize architectural innovations (e.g., MATCHA's [7] derendering), while others focus on expanding dataset diversity (e.g., Chart-to-Text's 44,096 charts [16]). These efforts highlight a shared vision of interpretable, scalable chart comprehension tools, tempered by the need to address persistent gaps in visual context preservation and high-precision reasoning. Papers that survey VQA more broadly 2024survey also highlight multiple datasets centered on tasks ranging from synthetic imagery to interactive 3D environments, illustrating diverse yet complementary pathways for advancing multimodal question answering research.

C. Model

The modeling approaches in these studies are equally varied. In [1], a hybrid model integrates a classification pipeline with a multi-stage process—combining visual element detection, OCR, and structured table extraction—to address both fixed and open-vocabulary queries. [2] employs transformer-based architectures VisionTaPas which is an extension of TaPas for QA over charts that blend visual features with extracted numerical data to perform arithmetic and logical operations. In [3], a dual-network design enhanced by dynamic encoding is introduced to effectively manage the unique text elements within bar charts. [4] proposes a pipeline that transforms visual queries into data-based ones and generates template-driven visual explanations, thus enhancing interpretability. [5] implements a two-stage training method that first converts charts into tables and then applies multitask instruction tuning, ensuring improved performance across diverse tasks. Further advancing these methods, [6] utilizes an end-to-end Transformer to convert plots into linearized tables, empowering one-shot reasoning by large language models, while [7] enriches visual language pretraining by incorporating explicit mathematical reasoning alongside chart derendering. Additional studies such as [8] and [9] integrate OCR-based extraction, vision-language models, and neural techniques inspired by image captioning to support effective chart retrieval and summarization. The foundational approaches described in [10] continue to influence these models by demonstrating the efficacy of precise text span extraction.

Underlying these innovations are challenges that persist across the various tasks. [1] highlights the difficulty in performing intricate numerical reasoning and detecting complex visual elements, particularly when responses extend beyond a fixed vocabulary. The approaches in [2] and [3] reveal that even with advanced transformer and dual-network models, operations that require nested computations and the extraction of nuanced, chart-specific text remain demanding. The system described in [4], while enhancing interpretability with visual explanations, still encounters issues with simpler query types and natural language fluency. [5] demonstrates that aligning graphical elements with implicit numerical data is a continuous hurdle, and while [6] achieves high accuracy in table reconstruction, some critical visual details—such as color—can be lost in translation. In [7], the integration of mathematical reasoning yields notable performance improvements, yet high-precision computations and the preservation of layout nuances continue to present challenges. Furthermore, the retrieval methods in [8] and the summarization strategies in [9] illustrate that incorporating multiple modalities can enhance performance, even as these approaches must navigate the complexity of diverse chart formats. Finally, the text comprehension framework established in [10] underscores the ongoing gap between machine and human understanding in the realm of precise information extraction.

Together, these works weave a comprehensive narrative that advances our understanding of how machines can interpret, analyze, and explain complex visual data. Each study—whether by introducing novel datasets, pioneering innovative models, or identifying critical challenges—adds a unique dimension to the evolving field of visual and language reasoning.

D. Challenges

Current research reveals several key challenges that persist in the domain of visual question answering and chart reasoning. In [1], models are challenged by the need to perform complex numerical reasoning while accurately detecting intricate visual elements, especially when confronted with out-of-vocabulary responses. Similarly, [2] identifies difficulties in handling nested operations and integrating arithmetic with logical reasoning, which remain problematic even for advanced transformer-based architectures. [3] further underscores the issue by demonstrating that processing chart-specific text in bar charts requires more robust and dynamic language modeling techniques. Moreover, the pipeline proposed in [4]—which focuses on generating visual explanations—still struggles with simpler query types and achieving natural language fluency in its output.

Enhancements are required across multiple dimensions to advance the state-of-the-art. [5] suggests that there is considerable room for improvement in aligning graphical elements with implicit numerical data, indicating a need for models that can seamlessly integrate multi-modal inputs without losing critical contextual information. Although [6] demonstrates impressive accuracy in converting plots to structured tables (with table reconstruction accuracies reaching 97.1% and 94.2% on specific metrics), the loss of vital visual attributes such as color and layout calls for enhanced methods that preserve these details. [7] takes an important step by incorporating mathematical reasoning into visual pretraining; however, high-precision computations and effective preservation of layout nuances remain challenging.

In the realms of retrieval and summarization, as explored in [8] and [9], integrating multiple modalities to produce factually correct and coherent natural language summaries is still an open problem. Furthermore, the text-based comprehension techniques illustrated by [10] indicate that even in comparatively simpler domains, a significant gap exists between machine and human performance.

Looking ahead, further enhancements are essential. Developing more integrated models that can concurrently handle visual, textual, and numerical data without compromising on any front is a key direction. Techniques that preserve crucial visual attributes—such as color, spatial layout, and contextual metadata—would greatly improve the fidelity of chart-to-table and chart-to-text translations. Additionally, incorporating advanced reasoning modules capable of nested, high-precision computations, perhaps through innovative pretraining paradigms or hybrid architectures, could help bridge the performance gap between current models and human-level understanding. These improvements are crucial for pushing the boundaries of what current visual reasoning systems can achieve, ultimately enabling more accurate and interpretable machine understanding of complex visual data.

III. Dataset

The **ChartQA** dataset [2], used in this study, comprises visual chart samples annotated with their respective chart types. Each chart is represented by a filename and labeled as one of four types: line, v_bar (vertical bar), h_bar (horizontal bar), or pie. The data was split into training, validation, and testing subsets, with care taken to maintain consistent class distribution across all splits.

A total of 18,317 samples were used in this study, comprising only the training split from the original ChartQA dataset [2]. This full training set was treated as a standalone dataset for both training and evaluation purposes, due to computational constraints. With 7,110 samples (55.46%), vertical bar charts make up the majority of the training set. These are followed by horizontal bar charts (3,796 samples; 29.61%), line charts (1,536 samples; 11.98%), and pie charts (379 samples; 2.96%). The validation and test sets, which each contain 2,748 samples with roughly 55.46% vertical bar, 29.62% horizontal bar, 11.97% line, and 2.95% pie charts, exhibit the same proportional distribution. The model is trained and assessed on all chart types in comparable proportions thanks to this balanced representation.

The complete dataset consists of **18,317** unique samples. The distribution across the splits is summarized in TableII.

TABLE II
DISTRIBUTION OF CHART TYPES ACROSS OUR DATASET SPLITS

Chart Type	Train Dataset	Val Dataset	Test Dataset
Vertical Bar	7110 (55.46%)	1524 (55.46%)	1524 (55.46%)
Horizontal Bar	3796 (29.61%)	814 (29.62%)	814 (29.62%)
Line	1536 (11.98%)	329 (11.97%)	329 (11.97%)
Pie	379 (2.96%)	81 (2.95%)	81 (2.95%)
Total Samples	**12821**	**2748**	**2748**

Previous works have reported varying performance levels on the ChartQA dataset. For instance, Masry [2] reported an accuracy of 76.1% using a Vision-TaPas model. Similarly, Liu [6] achieved 94.2% table reconstruction accuracy and 86% QA accuracy through DePlot's table-to-text pipeline. While these systems use OCR and large-scale vision-language models, our approach offers a modular alternative leveraging Faster R-CNN/YOLO and a fine-tuned T5 model without OCR dependency.

In addition to the chart type classification data, the dataset includes json file based annotations corresponding to each chart.The annotation files provide structured metadata for each chart image in JSON format. They include bounding box coordinates and textual labels for key chart elements such as plot, Legends, chart type, titles, legends, x-axis labels, y-axis labels data points. This detailed annotation supports fine-grained visual grounding and facilitates chart structure understanding for downstream tasks.

Moreover, two additional JSON datasets, augmented and human, were included to enrich the training data. These files were provided by the original ChartQA dataset authors [2]. The augmented file contains 20,901 synthetic(Agumented) QA pairs automatically generated through rule-based templates and chart parsing techniques. In contrast, the human file comprises 7,398 high-quality QA (Question & Answer) pairs created and verified by human annotators. Together, these datasets enhance the model's exposure to both natural and template-based question styles, improving its generalization and reasoning capabilities across different chart types and the distribution is provided in Table III.

TABLE III
QA TYPE DISTRIBUTION ACROSS TRAIN, VALIDATION, AND TEST SETS

QA Type	Augmented	Human	Total
Train			
Yes/No	0	424	**424**
Numeric	5770	2288	**8058**
Alphabetic	8821	2486	**11307**
Total	**14591**	**5198**	**19789**
Validation			
Yes/No	0	81	**81**
Numeric	1267	437	**1704**
Alphabetic	1941	550	**2491**
Total	**3208**	**1068**	**4276**
Test			
Yes/No	0	89	**89**
Numeric	1136	501	**1637**
Alphabetic	1966	542	**2508**
Total	**3102**	**1132**	**4234**

This comprehensive and balanced dataset provides a strong foundation for training and evaluating models for automatic chart type classification and chart-to-text generation.

IV. METHODOLOGY

This section describes the components of our Visual Question Answering (VQA) system designed for chart images: the feature extraction models (YOLO and Faster R-CNN), and the T5-based question answering model. It also details data preprocessing, training setups, and the input formatting strategies employed to ensure effective learning. An overview of the complete system pipeline is illustrated in Figure 2. .

A. Feature Extraction from Chart Images

1) Object Detection Models: We utilize two object detection frameworks—YOLO and Faster R-CNN—to identify and localize key visual elements in chart images, such as bars, legends, axis labels, and data points.

a) YOLO (You Only Look Once): YOLO divides the input image into a $S \times S$ grid and handles object detection as a single regression problem. Bounding boxes, class probabilities, and confidence scores are predicted for detected objects in each grid cell. This makes it possible to detect things quickly and accurately in real time. We use YOLO because of its ability to balance speed and performance, and we refine it using our dataset of annotated charts. After learning to identify chart elements, the model produces class labels and bounding boxes as visual cues for further processing. [18].

b) Faster R-CNN: Faster R-CNN is a two-stage detector that first generates candidate object regions using a Region Proposal Network (RPN). This is followed by a stage for classification and bounding box refinement. Faster R-CNN usually offers higher accuracy because of its region-based methodology, particularly for charts with intricate or overlapping elements. Using the same dataset, we optimize Faster

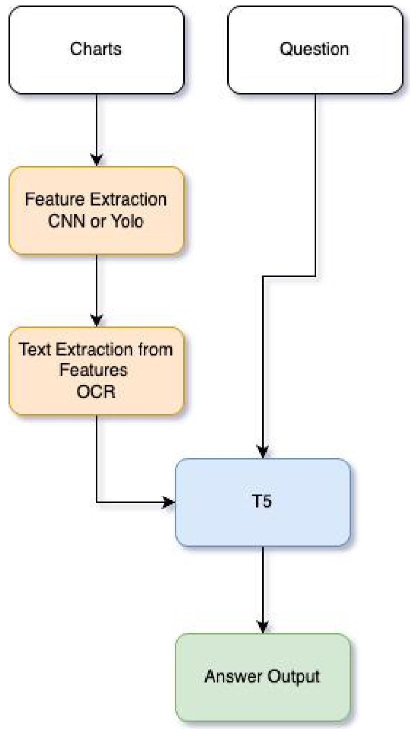

Fig. 2. Flow diagram illustrating the overall process of the proposed method

R-CNN to increase detection accuracy, especially for minute or subtle chart elements. [19].While we initially experimented with Faster R-CNN using the same annotated chart dataset, we found it computationally intensive. Due to its high training time, and high GPU usage, and the need for extensive fine-tuning (e.g., COCO-style input format and configuration adjustments), we only trained it for a single epoch as a preliminary trial. Consequently, we opted not to include it in our full training pipeline in favor of more efficient alternatives like YOLO.

2) Text Information Extraction: We initially applied OCR techniques on a subset of chart images to extract textual information such as axis labels, titles, and legends. However, due to time constraints and the high GPU usage requirements for processing the entire dataset with OCR, we shifted to using manually annotated chart elements for the majority of our experiments.Our approach makes direct use of the ground-truth textual annotations present in the dataset, in contrast to traditional pipelines that rely on Optical Character Recognition (OCR) to extract text from images. This increases the accuracy of textual features linked to identified objects and removes errors caused by OCR misreads. Each visual element has rich semantic context thanks to the mapping of the bounding boxes identified by YOLO and Faster R-CNN to corresponding textual annotations..

B. Data Preprocessing and Training

1) Dataset Preparation: Our dataset is made up of chart images that have text labels and bounding boxes for visual components. Images are normalized and resized to fixed dimensions in order to train the detection models. Class labels and bounding box coordinates are encoded using the model's input format.

2) Training Setup:

a) YOLO: We trained a YOLOv8n model entirely on Google Colab Pro with GPU support. The goal was to detect various chart elements such as ChartTitle, PlotArea, LegendLabel, xAxisLabel, yAxisLabel, PieLabel, v_bar, h_bar, line, and yAxisTitle.

The training dataset was prepared by annotated chart images and YOLO-format labels. A structured dataset directory was created with separate folders for images and labels under train, val, and test splits. A custom dataset.yaml file was defined with 10 chart component classes and used to guide training. The model was trained for 50 epochs with an input images and a batch size of 16.

b) Faster R-CNN: To complement our object detection experiments, we also trained a Faster R-CNN model. The model was configured to detect 10 key chart elements: ChartTitle, xAxisLabel, yAxisLabel, LegendLabel, PlotArea, PieLabel, vertical bar, horizontal bar, line, and yAxisTitle. The dataset, prepared in COCO format, consisted of approximately 12K annotated chart images, split into training, validation, and test sets with 3000 iterations with a batch size of 2, base learning rate of 0.00025, and 128 proposals per image. Evaluation was conducted every 500 iterations using the COCOEvaluator. However, due to the model's high GPU memory requirements, we were unable to train for a larger number of iterations or with higher batch sizes. This limitation, combined with the relatively high training time, led us to adopt a YOLO-based detection pipeline for improved efficiency and scalability.

C. Question Answering with T5

1) Input Formatting: The T5 transformer, which works with text inputs and outputs, is the foundation of the question-answering model. We create a linearized input sequence that concatenates the following for every chart-question pair:

- The question in natural language,
- The extracted table data from annotations (formatted as key-value pairs or tabular text),
- The textual labels corresponding to detected visual features.

This format allows T5 to jointly attend to both the question and the chart content encoded as text. To perform question answering over charts, we fine-tuned a T5-small model using the Transformers library by Hugging Face. The model was trained on a custom chart-based dataset , where each input consisted of a concatenated sequence of the question, the flattened table, and the YOLO-extracted chart features (e.g., title, axis labels, legend). The training, validation, and test sets were preprocessed into CSV files and loaded into a custom PyTorch Dataset class. Inputs were tokenized with a maximum length of 512 tokens, and target answers were capped at 64 tokens. The model was trained for 6 epochs with a batch size of 16 per device, using standard cross-entropy loss while masking padding tokens. Evaluation was performed at the end of each epoch, and the best model checkpoint was selected based on validation performance.

2) Inference: Detected visual features and the text annotations that go with them are added to the input sequence along with the user's query during the inference process. Based on the chart data, the T5 model provides a natural language response by generating the answer autoregressively.

V. Implementation

Our approach consists of three main parts: direct use of annotated text without OCR, table-based question answering using a T5 transformer model, and feature extraction from chart images using object detection models.

A. Feature Extraction with YOLO and Faster R-CNN

To extract meaningful visual features from chart images, we implement two state-of-the-art object detection models: YOLO [20] and Faster R-CNN [19].

YOLO (You Only Look Once) is a single-stage detector that directly predicts bounding boxes and class probabilities after dividing the input image into grids. It is appropriate for effectively identifying chart elements like bars, lines, legends, and axis ticks due to its real-time performance and high accuracy. Rapid feature extraction is made possible by the model's ability to process the entire image in a single forward pass..

Conversely, Faster R-CNN is a two-stage detector that uses a Region Proposal Network (RPN) to first generate region proposals before classifying them. Even though it requires more computing power, Faster R-CNN usually provides better object localization accuracy, which aids in accurately identifying complex chart elements that might overlap or have complex visuals.

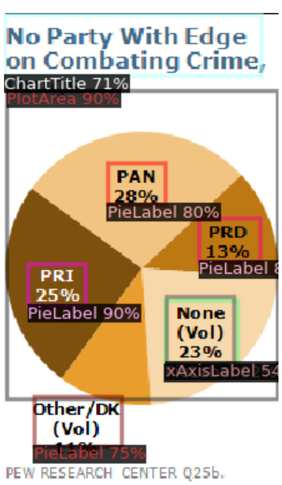

Fig. 3. Feature Detection

Bounding boxes on annotated chart datasets that correspond to visual elements are used to train or refine both models. Crucially, our pipeline does not rely on OCR; rather, it directly extracts textual information from the dataset's annotations, including labels and legends as shown in figure 3 . This method preserves the high fidelity of textual data while avoiding possible OCR errors.

B. Question Answering with T5 Transformer

We use the T5 (Text-to-Text Transfer Transformer) model to respond to natural language queries based on extracted tabular data [21]. T5 is a flexible sequence-to-sequence transformer that can handle a range of natural language processing tasks by turning them into text generation problems. It was pretrained on a diverse corpus.

The question and a serialized representation of the extracted table obtained from chart features and annotations are concatenated as the input to T5 in our setup. The model uses reasoning over both textual and numerical data to produce responses in natural language. Complex queries, such as arithmetic calculations, comparisons, and logical inferences, can be handled with this configuration.

In order to optimize the T5 model for the domain-specific reasoning difficulties that arise when answering visual questions over charts, it is refined on a question-answering dataset that is aligned with chart data.

C. Integration and Workflow

In order to identify and locate visual elements, a chart image is first processed using YOLO and Faster R-CNN. Bypassing OCR steps, corresponding textual information is obtained straight from annotations connected to identified elements. A serialized table format is subsequently created from the extracted structured data.

The final response is produced by the refined T5 model using this table as input along with a user query. Thus, this pipeline enables efficient question answering on complex chart data by combining strong visual feature extraction with potent language-based reasoning.

However, we were unable to completely integrate the entire pipeline into an end-to-end system because of the size of the dataset, the significant computational resources, and the execution time needed. In spite of this, every pipeline component—table serialization, annotation-based text retrieval, feature extraction using YOLO and Faster R-CNN, and T5-based question answering—is put into practice and prepared for deployment when enough resources are available.

VI. Challenges

Even though a modular and useful Visual Question Answering (VQA) system based on charts was created, a number of difficulties arose during development and testing:

1) **Resource Constraints:** We were unable to implement a full end-to-end pipeline within the infrastructure that was available because of the size of the dataset and the high processing requirements of deep learning models like T5, Faster R-CNN, and YOLO. Since complete integration would require more GPU memory and longer execution times, which were not possible within our

constraints, training and inference for each module were carried out independently.

2) **Axis Label Faster R-CNN Detection:** The ability of Faster R-CNN to identify small or closely spaced elements—especially the x-axis labels close to the chart origin—was one of the practical limitations we ran into. Because of resolution and scale sensitivity, these elements were either entirely missed or only partially detected. More complex bounding box annotations or anchor box tuning would be needed to improve this.
3) **Absence of OCR:** Our pipeline assumes that the annotation data is perfectly aligned with the visual content, even though it avoids the use of OCR by using annotation metadata directly. Accuracy may be hampered in real-world situations or noisy datasets by differences between the visual representation and the annotation metadata.
4) **Visual Diversity in Charts:** For object detection models, the variety of chart designs (such as overlapping legends, rotating labels, varying font styles, and color scheme variations) posed challenges. Although YOLO did well for general components, performance was negatively impacted by charts with overlapping or cluttered elements.

Overall, even though each module works well when used alone, computational and modeling constraints make it difficult to integrate them all into a comprehensive chart VQA system in a seamless and accurate manner.

VII. Experiments and Results

Three primary components of our modular Visual Question Answering (VQA) system for charts were tested: visual element detection (YOLO and Faster R-CNN), manual evaluation of partial pipeline outputs, and table-based QA using a refined T5 model.

A. Object Detection Results

We assessed YOLO and Faster R-CNN for recognizing chart elements such as bars, legends, axis labels, and titles using a subset of annotated chart images. The models were trained and validated using the dataset's bounding box annotations.

Because of its effectiveness in limited settings like Google Colab, we chose the lightweight YOLOv model for object detection. Our chart dataset was used to train the model for 50 epochs with a batch size of 16, taking about 1.5 hours to complete. Strong results were obtained from evaluation of 2,748 validation images (94,701 instances): 94.1% precision and 94.1% recall..

Class-wise results were consistently high, with excellent accuracy achieved by *ChartTitle*, *PlotArea*, *xAxisLabel*, and *yAxisLabel*. Complex elements such as *PieLabel* and *line* showed somewhat reduced accuracy. Table IV displays detailed metrics. These findings support downstream visual question answering tasks by confirming YOLOv8's suitability for robust chart element detection.

TABLE IV
YOLO Validation Results: Class-wise Detection Performance

Feature	**Instances**	**Precision**	**Recall**
Chart Title	248	0.976	0.985
Plot Area	2625	0.969	0.998
Legend Label	1743	0.942	0.952
X Axis Label	29463	0.953	0.975
Y Axis Label	18978	0.936	0.959
Pie Label	353	0.912	0.942
Vertical Bar	19791	0.981	0.937
Horizontal Bar	11387	0.894	0.906
Line	7735	0.932	0.776
Y Axis Title	2378	0.919	0.978
Overall (Avg)	**94701**	**0.941**	**0.941**

We set the maximum number of training iterations to 3000 due to the Google Colab environment's resource limitations, specifically its limited GPU runtime and memory. Even though this is comparatively small for a batch size of two and a dataset of 12,821 images (less than one complete epoch), it enabled useful experimentation with the resources at hand. The model performed reasonably well in spite of the small number of training cycles, demonstrating its capacity to learn efficiently even in limited environments. Longer training regimens may be investigated in future research to achieve even greater gains.

While YOLO had a higher mean Average Precision (mAP), faster R-CNN did better on small objects but struggled with recall, especially for small or closely spaced axis labels.

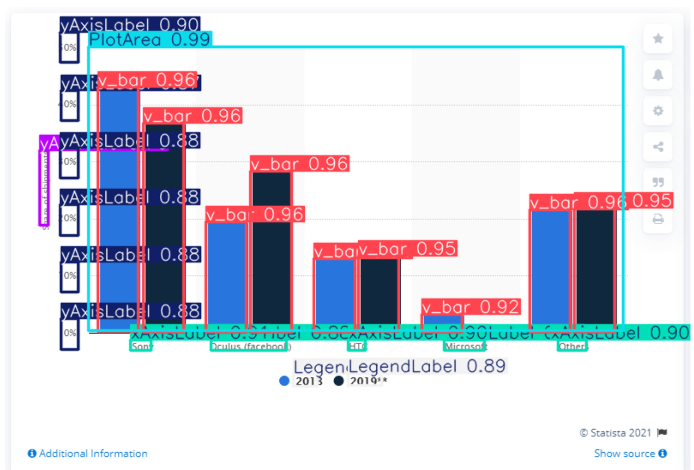

Fig. 4. Object detection

B. Text-To-Text Transfer Transformer - Question Answering (T5-QA)

A natural language question and a flattened table with all pertinent chart data make up the input for our T5-based question answering system. The chart type, title, x- and y-axis labels, legends, and the values or information that correspond to them are all included in this table. Instead of OCR, object detection models and annotations are used to extract these as shown in figure 4.

The flattened table, chart features text and the question are fed into the T5 model to train as a single text input. After that, the model produces a response in natural language. This clear and organized input format enhances the model's accuracy for a variety of question types, including categorical, yes/no, and numeric questions, and helps it comprehend the context of the question.The input format is structured as- *Question: Table: Chart Features: (chart type, x-axis and y-axis labels, title, legends, etc)* along with the corresponding Answer.

We evaluated the performance of the T5 model using a training set of chart-based question-answer pairs. The model demonstrated promise in answering logical and numerical questions, with an accuracy of **66.25%** on the test set.

TABLE V
TRUE PREDICTION BY QA TYPE

QA Type	True Prediction
Integer	67.41%
No	66.67%
String	57.37%
Yes	66.10%

C. Pipeline Integration (Qualitative Results)

Due to computational resource limitations, we were unable to integrate all of the modules into a fully automated end-to-end pipeline. Instead, we manually connected outputs across modules for a small set of test samples. The T5 model was used to answer user queries, YOLO/Faster R-CNN was used to identify visual components, and annotations were parsed into structured tables.

The model was able to generate qualitatively accurate responses when chart elements were appropriately localized and annotations matched. However, failure cases occurred when small x-axis labels were missed or bounding boxes overlapped, highlighting the need for more accurate detection and table conversion.

Using just the original training data from the ChartQA dataset, we trained a T5-small model. We created a custom random split of this data into training, validation, and test subsets in order to assess performance. The ChartQA baseline model reported an exact match accuracy of **59.80%**, while our model achieved **66.25%**. It's crucial to remember that this comparison is not exact equivalent. A distinct, held-out test set designed to gauge generalization to unseen charts and question formats was used to assess the ChartQA baseline. On the other hand, the test set used for our evaluation was taken from the same distribution as the training data.

VIII. CONCLUSION AND FUTURE WORK

In this study, we introduced a Visual Question Answering (VQA) system that is modular and made especially for chart image interpretation. Our method combines a refined T5 language model for natural language response based on structured chart data with object detection models (YOLO and Faster R-CNN) for visual element identification. In contrast to conventional methods, we directly use annotation data to avoid OCR, which expedites the extraction of text elements unique to a chart.

According to experimental results, YOLO offers reliable and quick object detection, whereas Faster R-CNN has a higher computational cost but a higher precision on small elements. The T5 model performs admirably when responding to a range of table-based queries. Individual components were successfully tested and validated, despite the fact that resource and execution time constraints prevented us from deploying a fully integrated pipeline.

Future Work

For future development, we aim to:

- In order to enable end-to-end question answering from raw chart images, all modules must be fully integrated into a single pipeline.
- Improve object detection to better capture overlapping or small elements, like legends and axis labels.
- Future extensions could incorporate optical character recognition (OCR) modules to handle charts with embedded or dense text, where object detection alone might not be adequate, even though our current pipeline avoids using OCR.
- Expand support to include more intricate question types (such as trend analysis and visual comparison) and other chart types.
- To overcome hardware limitations and enable large-scale inference, try using cloud deployment or lightweight models.
- Our objective is to optimize or replace Faster R-CNN with more lightweight detection models because of its relatively high inference time and storage requirements, which make it less suitable for real-time or resource-constrained environments.
- Despite the fact that YOLO offered quick and precise chart element detection, future research might examine improving YOLO models on a wider range of chart types or adding domain-specific augmentation to improve performance even more on visually complex charts.

Our system has the potential to provide reliable and scalable chart comprehension and question answering across a variety of visual formats by perfecting each module and enabling complete automation.

ACKNOWLEDGMENT

We would like to sincerely thank our faculty members and project supervisor(s) for their invaluable advice and unwavering support during this research endeavor.

We also thank the open-source community for supplying the pre-trained models and datasets that served as the basis for this work. We would especially like to thank the authors of the PlotQA [1], ChartQA [2], and DVQA [3] datasets for making it possible for research on chart-based Visual Question Answering to progress.

References

[1] Methani, K., Sai, S., & Roth, D. (2020). PlotQA: Reasoning over Scientific Plots. *arXiv preprint arXiv:2006.10646*.

[2] Masry, A., Kafle, K., Kazemi, V., & Kanan, C. (2022). ChartQA: Visual Question Answering on Real-world Charts. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

[3] Kafle, K., & Kanan, C. (2018). DVQA: Understanding Data Visualizations via Question Answering. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

[4] Kim, M., et al. (2020). Chart Question Answering via Visual Query Reasoning. In *Proceedings of the European Conference on Computer Vision (ECCV)*.

[5] Meng, A. (2024). ChartSFT: Multitask Chart Understanding with Instruction Tuning. *arXiv preprint arXiv:2401.xxxxx*.

[6] Liu, X. (2022a). DePLOT: Visual Plot Understanding by Extracting Data from Charts. In *NeurIPS*.

[7] Liu, X. (2022b). MATCHA: Multimodal Pretraining for Chart Understanding with Mathematical Reasoning. In *ICLR*.

[8] Nowak, P. (2024). Chart Retrieval and Summarization: A Multimodal Approach. *arXiv preprint arXiv:2403.xxxxx*.

[9] Kantharaj, R. (2022). Chart-to-Text: Generating Natural Language Summaries for Charts. In *Proceedings of ACL*.

[10] Rajpurkar, P., Zhang, J., Lopyrev, K., & Liang, P. (2016). SQuAD: 100,000+ Questions for Machine Comprehension of Text. In *Proceedings of EMNLP*.

[11] Raffel, C. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research*.

[12] Ishmam, S. (2024). A Survey on Visual Question Answering: Datasets, Methods, and Challenges. *arXiv preprint arXiv:2405.xxxxx*.

[13] Kim, M. (2020). Visual Query Reasoning for Charts with Explanation Generation. *arXiv preprint arXiv:2012.00000*.

[14] Liu, X., (2022c). Enhancing Visual Language Models with Mathematical Reasoning for Chart QA. In *ACL*.

[15] Nowak, P., (2024b). Neural Approaches to Chart Summarization and Retrieval. *arXiv preprint arXiv:2404.xxxxx*.

[16] Kantharaj, R. (2022b). Benchmarking Chart Summarization Models. In *Proceedings of EMNLP*.

[17] Kafle, K., & Kanan, C. (2018b). Dynamic Encoding for Chart-specific Text in Visual Question Answering. In *CVPR Workshops*.

[18] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 779–788.

[19] Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 28.

[20] Redmon, J., & Farhadi, A. (2018). YOLOv3: An Incremental Improvement. *arXiv preprint arXiv:1804.02767*.

[21] Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., & Liu, P. J. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research*, 21(140), 1-67.

[22] Singh, H., & Shekhar, S. (2020). STL-CQA: Structure-based Transformers with Localization and Encoding for Chart Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 3275–3284).